

BITCOIN++ BERLIN 2026 · PAYMENTS EDITION

Sats4Tokens

Sats in, tokens out.

Pay with ⚡ Lightning or 🥜 Cashu ecash, get an AI API key for Claude, GPT, DeepSeek & more. No signup, no KYC. Every payment credited **exactly once**, even if the server is killed mid-payment.

⚡ Lightning

🥜 Cashu ecash

NUT-09 recovery

mainnet



Pay 1,161 sat

any Lightning wallet, or paste ecash



Settled exactly once

write-ahead · NUT-13 · NUT-09 restore



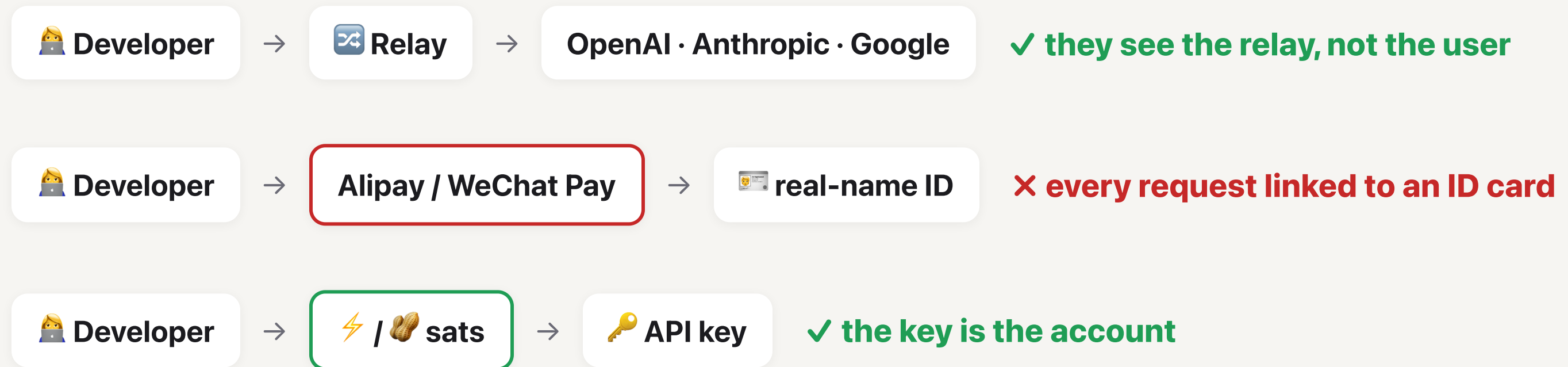
Your API key, capped at \$1

sk-..... · /v1 · Claude Code

THE PROBLEM

My AI relay is a privacy layer. Payment breaks it.

I run an AI API relay: **310 real users**, ~12 billion tokens a month. One OpenAI-compatible key for every model.



Sats4Tokens: pay a Lightning invoice or paste Cashu ecash, get a capped API key. No name, no email, no card, and **never charged twice**, even if the server dies mid-payment.

Pay sats → get a key. No signup.

Real UI · key masked
numbers from a real mainnet order

1 Pick \$1 / \$2 / \$5 / \$10

 **Sats4Tokens**

Sats in, tokens out. Pay with bitcoin, get an AI API key — no signup, no email, no KYC. · 付款即得 key, 无需注册

One key works with Claude, GPT, DeepSeek, GLM, Kimi ... through one OpenAI-compatible endpoint (and the Anthropic API for Claude Code). The key is limited to exactly what you pay.

Buy a \$1 key

Buy a \$2 key

Buy a \$5 key

Buy a \$10 key

Pay with ⚡ Lightning or 🥜 Cashu ecash. Keep the next page — it is your receipt and shows your key.

2 Pay: ⚡ Lightning or 🥜 Cashu

 **Pay with Bitcoin**

No account. No KYC. No card. · 无需注册, 无需实名

AI API key · \$1 · order CKPAYDEM00...

1,161 sat
= 1 USD · 1 BTC = 86,181 USD

⚡ Lightning

🥜 Cashu ecash



3 The key is the account

 **Paid**

1160 sat received via cashu.

 **Your API key**

Limited to what you paid. No account — this key is the account. Save it (or keep this page's URL).

OPENAI_BASE_URL=https://sats4tokens.bhbtc.
xyz/v1
OPENAI_API_KEY=sk-
.....

Copy

Works with any OpenAI-compatible app. Test it:

curl
https://sats4tokens.bhbtc.xyz/v1/chat/comp
letions \

THE RECEIPT IS THE KEY

One key, every model.

- **OpenAI-compatible**
any app, one endpoint
- **Claude Code one-liner**
copy, paste, run
- **Usage, call by call**
what the key spent, live
- **Prices for every model**
USD per 1M tokens
- **Capped at what you paid**
no card on file, no overdraft

Or run **Claude Code** on this key — paste in a terminal:

```
ANTHROPIC_BASE_URL=https://sats4tokens.bhbtc.xyz \  
  ANTHROPIC_AUTH_TOKEN=sk-  
..... \  
  ANTHROPIC_MODEL=claude-opus-4-8  
ANTHROPIC_DEFAULT_HAIKU_MODEL=claude-opus-4-6 \  
  
Copy
```

Usage \$0.898688 left · \$0.101312 used · 4 calls
[↻ Refresh](#)

Time	Model	Tokens in / out	Cost
12:02:25	claude-opus-4-8	947 / 7	\$0.100528
12:01:01	claude-opus-4-8	9 / 5	\$0.000148
12:00:59	claude-opus-4-6	9 / 5	\$0.000148
11:20:23	deepseek-v4-flash	31 / 44	\$0.000488

Models & prices

USD per 1M tokens. This key works for every model below. **Claude Code** = also speaks the Anthropic API.

Models & prices

USD per 1M tokens. This key works for every model below. **Claude Code** = also speaks the Anthropic API.
"fast x2": service_tier fast/priority costs double.

Model	Input	Output	Cached
Anthropic Claude Code			
claude-opus-4-6	\$3.9	\$19.5	\$0.39
claude-opus-4-6-kiro	\$3.9	\$19.5	\$0.39
claude-opus-4-7	\$3.9	\$19.5	\$0.39
claude-opus-4-7-kiro	\$3.9	\$19.5	\$0.39
claude-opus-4-8	\$3.9	\$19.5	\$0.39
claude-opus-4-8-kiro	\$3.9	\$19.5	\$0.39
claude-opus-5-5	\$10.9	\$54.48	\$0.545
claude-opus-5	\$13.62	\$68.1	\$1.36
claude-opus-5-max-all	\$16.25	\$81.25	\$1.63
claude-opus-4-8-max	\$16.88	\$84.42	\$1.69
claude-fable-5	\$27.24	\$136.19	\$2.72
claude-fable-5-1	\$27.24	\$136.19	\$2.72
DeepSeek			
deepseek-v4-flash	\$3	\$9	\$0.1
deepseek-v4-pro	\$9	\$27	\$0.3

HOW IT WORKS

No Lightning node. The mint is the node.



Exactly once

- 1. Outputs come from seed + counter (**NUT-13**): deterministic.
- 2. **Write first**, then call the mint: counter range → disk (fsync + atomic rename).
- 3. After a crash: **NUT-09 restore** of that exact range → one pure function decides →
- 4. Crediting is idempotent: key by order id, top-up once per order.

Recovery table — decideSettle(), first match wins

	FACTS AFTER RESTART	ACTION	WHY
1	restore found our outputs	finish	mint signed, we lost the reply
2	⚡ quote PAID	retry	same counters → same outputs
3	⚡ quote ISSUED / UNPAID	conflict	needs a human
4	🥜 inputs UNSPENT	retry	swap never happened
5	🥜 inputs PENDING	wait	mint still working
6	🥜 inputs SPENT	abort	token spent elsewhere

NPM RUN CRASH-DEMO

kill -9 mid-payment. Credited once.

1. The mint signs our ecash outputs...
2. ...and the gateway is **killed** before saving them.
3. Restart: **NUT-09 restore** of the counter range written *before* the mint call finds them.
4. Finish the order, credit **once**. Replaying the token is rejected.

4 cases (⚡/💰 × 2 crash points), all green.

Real output, testnut mint, ~10 s. Setup lines trimmed.

npm run crash-demo -- cashu after-mint

► merchant creates a \$1.00 top-up order

```
[gateway] 📄 CEMUQTPR0R13E788BD: 1.00 usd = 1000 sat
```

► customer pastes a cashu token → gateway swaps it at the mint ... and gets killed

```
[gateway] ✨ CRASH_AT=after-mint – killing the process
```

```
customer's request died with the gateway: SocketError: other side closed
```

```
gateway exited: SIGKILL
```

► ledger on disk right now

```
state=SETTLING write-ahead counters 1..7 balance proofs=0
```

```
merchant credited so far: []
```

► restart the gateway → recover() runs before it accepts requests

```
[gateway] 🔄 recover (cashu, counters 1..7):
```

```
{ restored: 7, inputState: 'SPENT' } → finish
```

```
[gateway] balance=1001 sat
```

```
[merchant] 💰 credited (1.00) trade_no=CEMUQTPR0R13E788BD
```

► result

```
order state=PAID via=cashu received=1001 sat notify done=true
```

```
merchant credited 1×
```

```
replaying the same token → 400 {"error":"order is paid"}
```

✅ killed mid-payment, recovered, credited exactly once

Paid from a phone wallet. Twice.

Key shop

\$1 = 1,161 sat

- Paid with Cashu ecash (Coinos mint)
- Key shown on the receipt page, capped at \$1
- Real calls: Claude Opus, DeepSeek; usage visible per call

Top-up for an existing user

\$1 = 1,158 sat

- Relay account balance credited
- Operator withdrew everything the gateway held as one Cashu token
- Received back in a wallet (proofs SPENT at the mint)

Built at the hackathon: ~1,750 lines TypeScript/HTML + 600 lines tests & scripts · Node 22 · 2 dependencies (@cashu/cashu-ts, qrcode)

Polite to public mints: NUT-17 WebSocket push first, polling only as a budgeted safety net (≤ 1 check / 8 s).

HONEST LIMITS

- The mint is custodial until I withdraw → withdraw often, or run my own mint
- One mint per gateway (pay with ⚡ from any other)
- Overpayment is kept: a pasted token isn't split for change

NEXT

- NUT-18 payment requests: scan instead of paste
- Auto-melt to the operator's Lightning wallet
- Top up an existing key
- Pay-per-request: Cashu tokens in the API header

Sats in, tokens out.

Private · no KYC · **exactly once**. Thank you!